ORACLE

Solving Zombie Cgroups – Simple Fix or Smart Solution?

Harry Yoo, Kamalesh Babulal

harry.yoo@oracle.com, kamalesh.babulal@oracle.com

Oracle Cloud Infrastructure

December 12, 2025

Safe harbor statement

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described for Oracle's products may change and remains at the sole discretion of Oracle Corporation.

Copyright © 2025, Oracle and/or its affiliates |

What is a zombie memory cgroup?

“cgroup is a mechanism to organize processes hierarchically and distribute system resources along the hierarchy in a controlled and configurable manner”

Documentation/admin-guide/cgroup-v2.rst

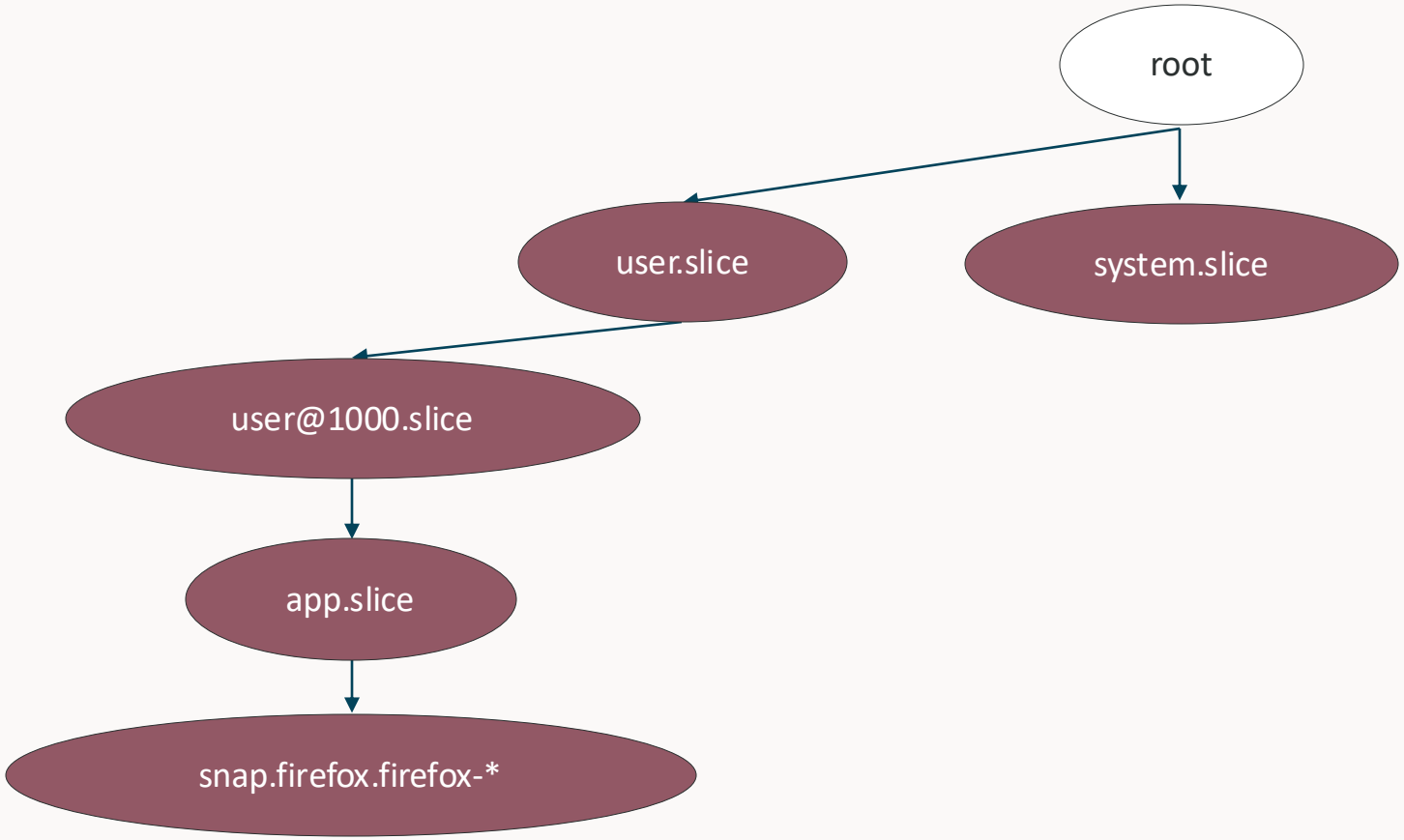
Kernel Documentation

* When I mention **memory cgroup** in this talk,
  it means cgroup that has memory controller enabled

Copyright © 2025, Oracle and/or its affiliates |

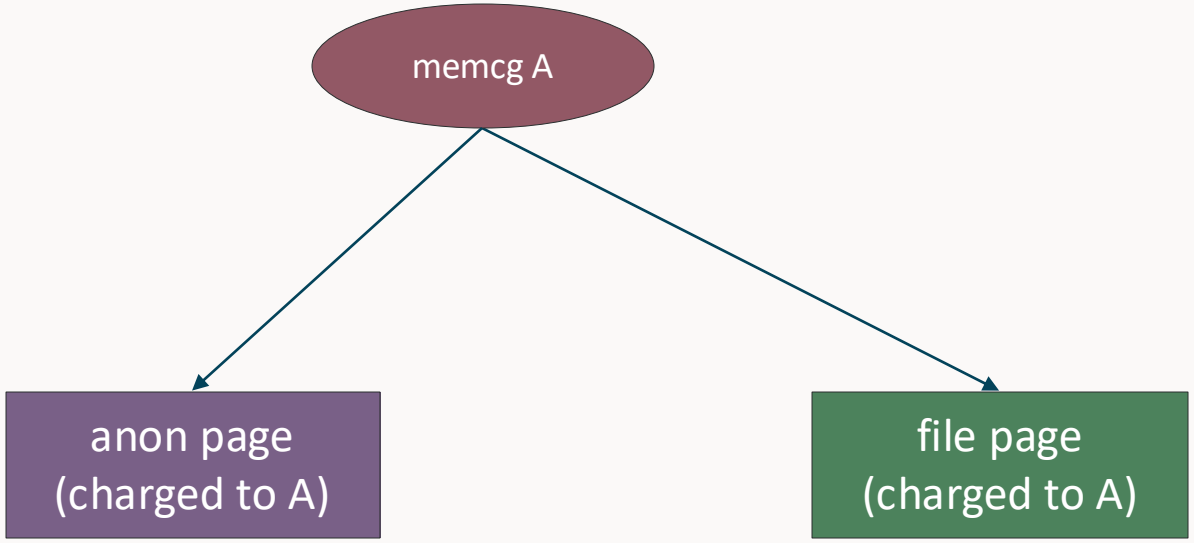
Cgroup is wide-spread in modern Linux systems

Part of the cgroup hierarchy on the speaker's desktop environment



Copyright © 2025, Oracle and/or its affiliates |

Memory cgroups track how much memory is allocated for cgroup

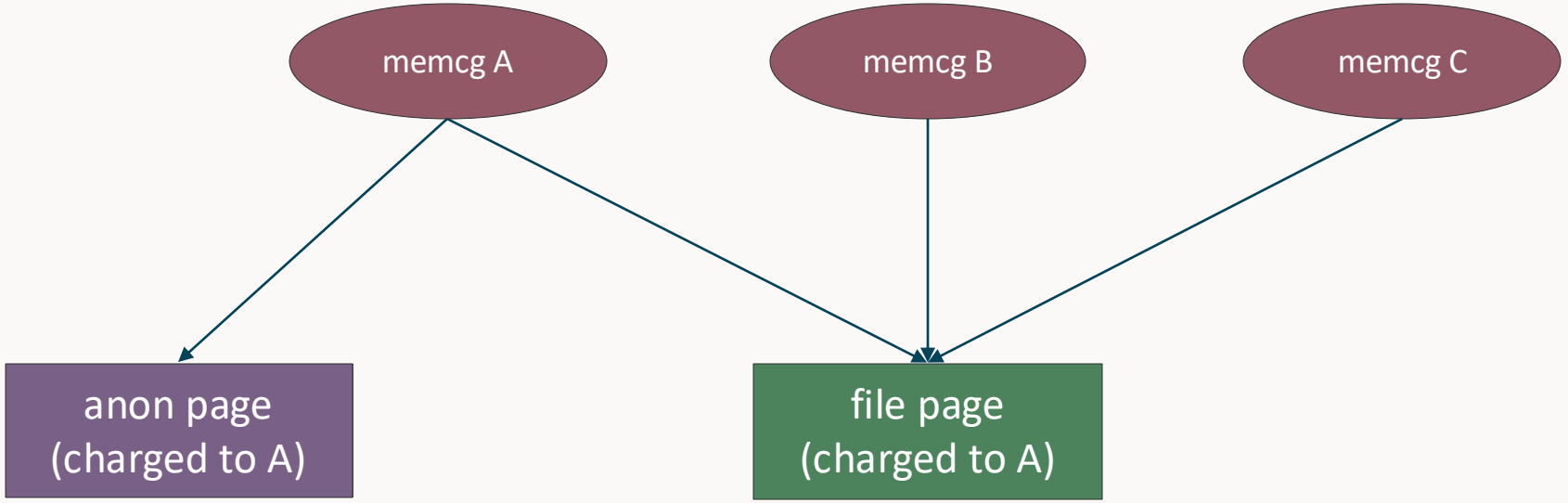


- memcg A allocates a file page (might be shared to other memcgs)
- memcg A allocates an anon page (private to memcg A)
- memcg A gets charged for both of them

Copyright © 2025, Oracle and/or its affiliates |

Shared pages are duly charged – only the first memcg

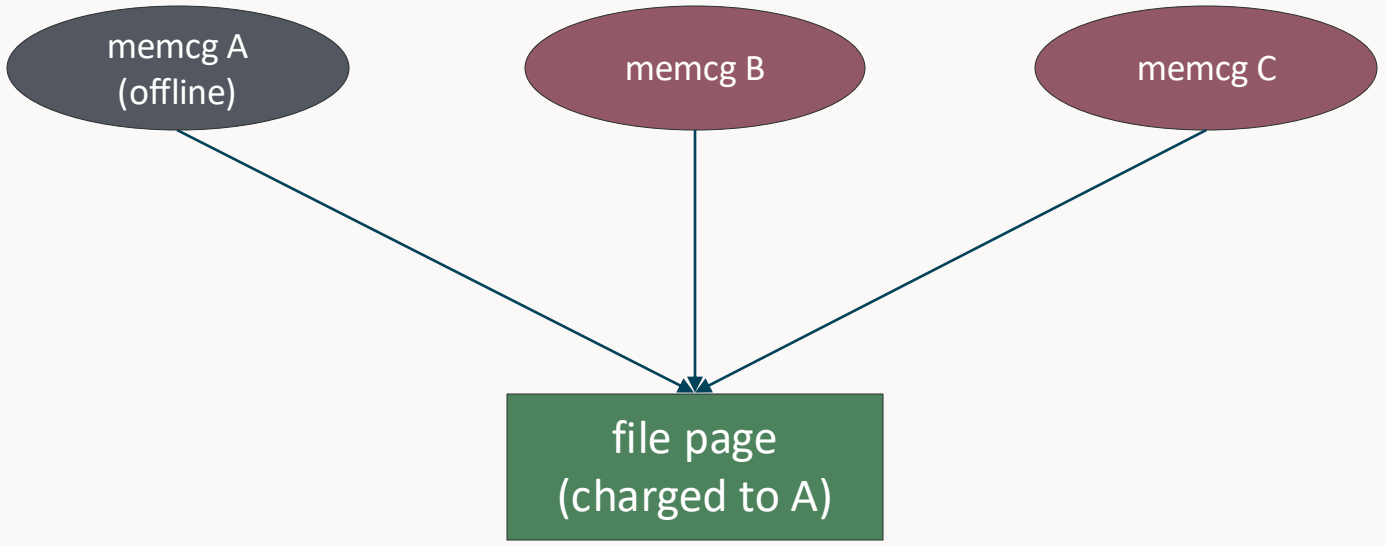
Memory controller is never designed to work well with shared resources



- memcg B, C accesses the file page, but don't get charged and access it for free
- When pages are shared, only the first memcg gets charged for it!

Copyright © 2025, Oracle and/or its affiliates |

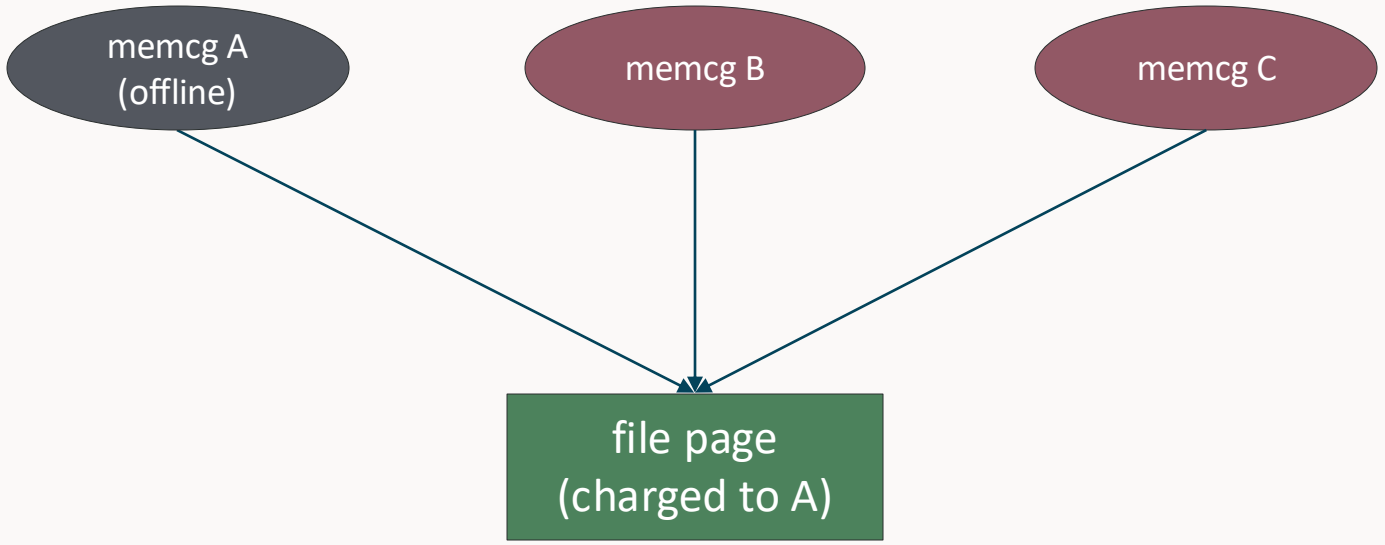
What happens when memcg A gets offlined?



- memcg A is removed from the hierarchy, but the file page is still charged to A
- memcg A cannot be released until the file page is reclaimed, **becomes a zombie**
- Unlike the file page, the anon page is reclaimed before offlining because it is not shared

Copyright © 2025, Oracle and/or its affiliates |

Zombies accumulate, memory gets wasted



Zombie Memcg Problem

- Memory cgroups becomes zombie due to their shared resources outlive memcg themselves
- As cgroups are created and destroyed, zombie memcgs accumulate and waste memory

Copyright © 2025, Oracle and/or its affiliates |

Least-Recently-Used Mechanism

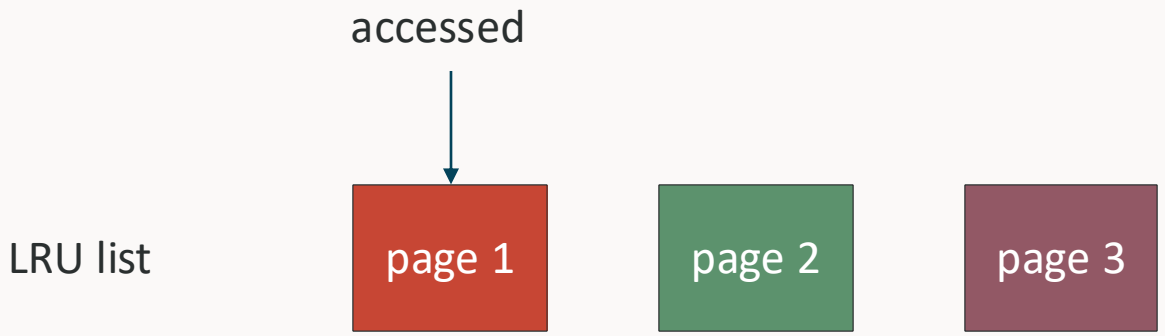
Least-Recently-Used sorts page by access time



- Conceptually, LRU mechanism sorts pages by access time
- In reality, we approximate LRU as sorting list for every memory access is too expensive

Copyright © 2025, Oracle and/or its affiliates |

Least-Recently-Used sorts page by access time



- Conceptually, LRU mechanism sorts pages by access time
- In reality, we approximate LRU as sorting list for every memory access is too expensive

Copyright © 2025, Oracle and/or its affiliates |

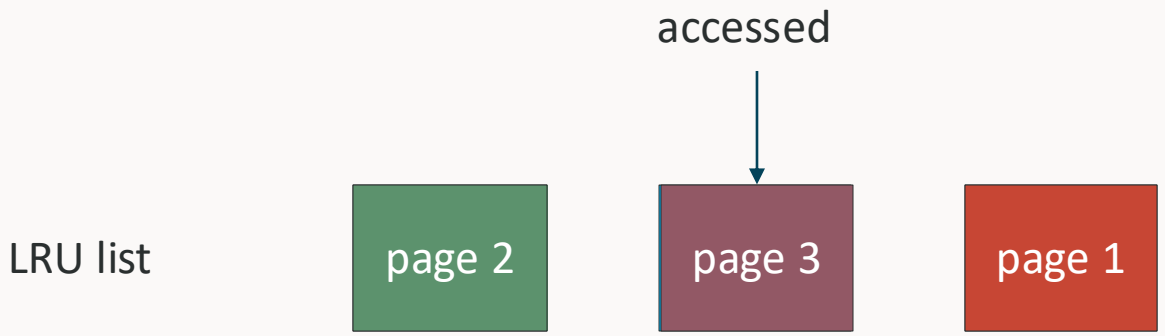
Least-Recently-Used sorts page by access time



- Conceptually, LRU mechanism sorts pages by access time
- In reality, we approximate LRU as sorting list for every memory access is too expensive

Copyright © 2025, Oracle and/or its affiliates |

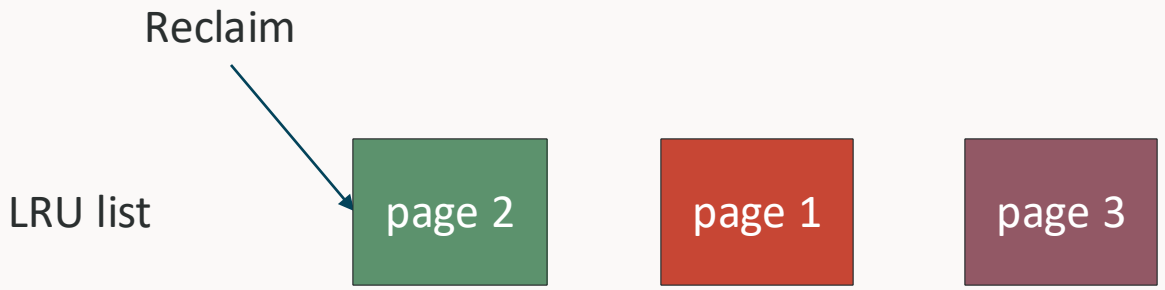
Least-Recently-Used sorts page by access time



- Conceptually, LRU mechanism sorts pages by access time
- In reality, we approximate LRU as sorting list for every memory access is too expensive

Copyright © 2025, Oracle and/or its affiliates |

Memory manager reclaims the least recently used page



- Conceptually, LRU mechanism sorts pages by access time
- In reality, we approximate LRU as sorting list for every memory access is too expensive

Copyright © 2025, Oracle and/or its affiliates |

Memory manager reclaims the least recently used page

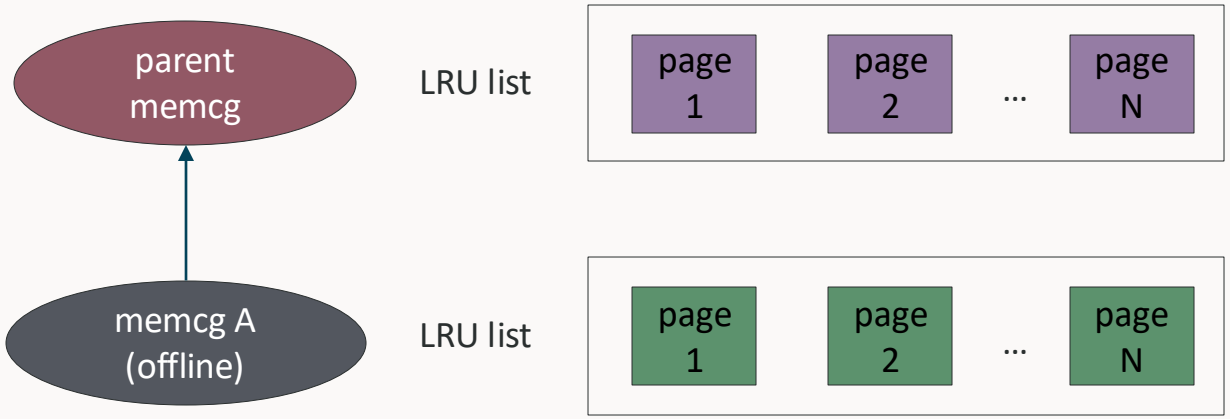


- Conceptually, LRU mechanism sorts pages by access time
- In reality, we approximate LRU as sorting list for every memory access is too expensive

Copyright © 2025, Oracle and/or its affiliates |

Reparenting to eliminate zombies

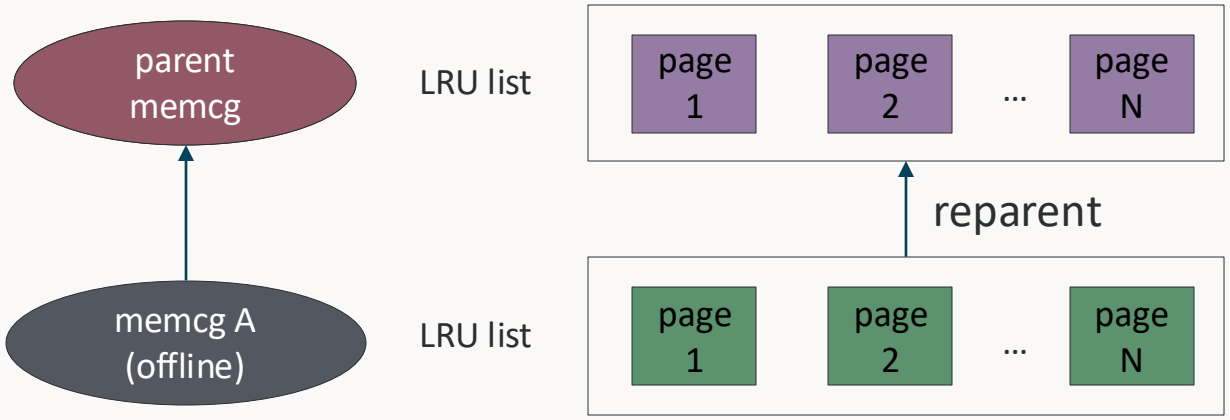
Zombie memcgs accumulate due to resources that outlive them



- Each memcg has its own LRU list
- w/o reparenting, memcg A cannot be released until all of its pages are reclaimed

Copyright © 2025, Oracle and/or its affiliates |

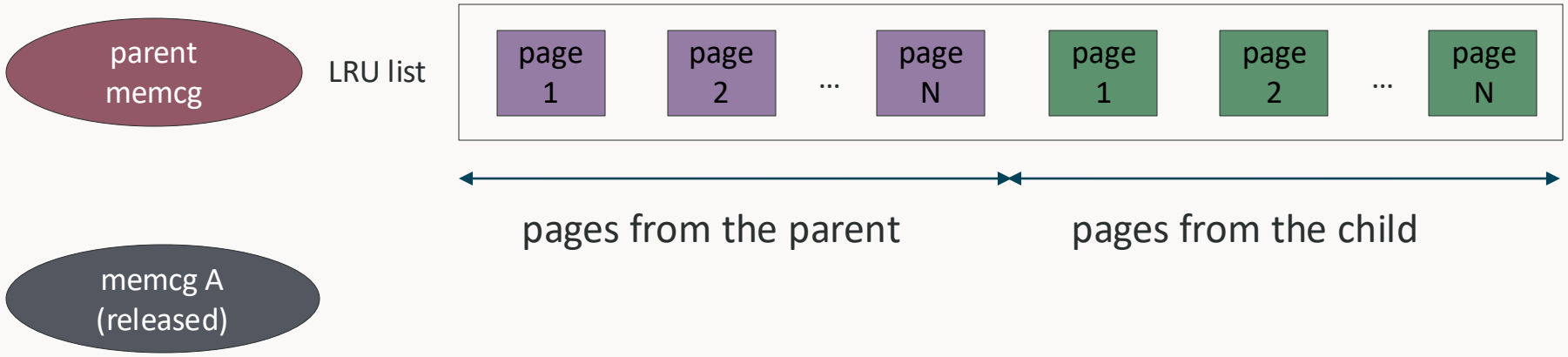
Reparenting: transfer resources to the parent



- Reparenting transfers resources to the parent memcg

Copyright © 2025, Oracle and/or its affiliates | Linux Plumbers Conference, Tokyo 2025

“Simple” Reparenting: splice LRU list into the parent’s



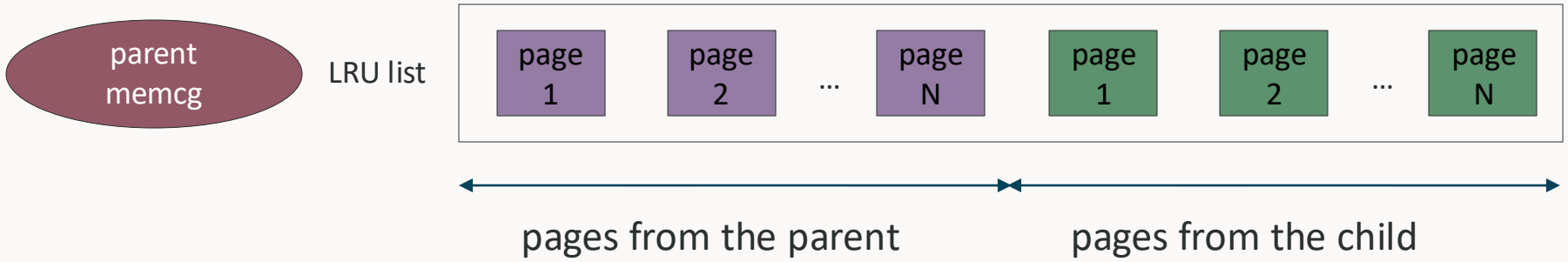
- “Simple” reparenting: splice child LRU list into parent’s, upon memcg offlining
- memcg A can be released after reparenting

Copyright © 2025, Oracle and/or its affiliates |

How can this disturb memory reclaim?



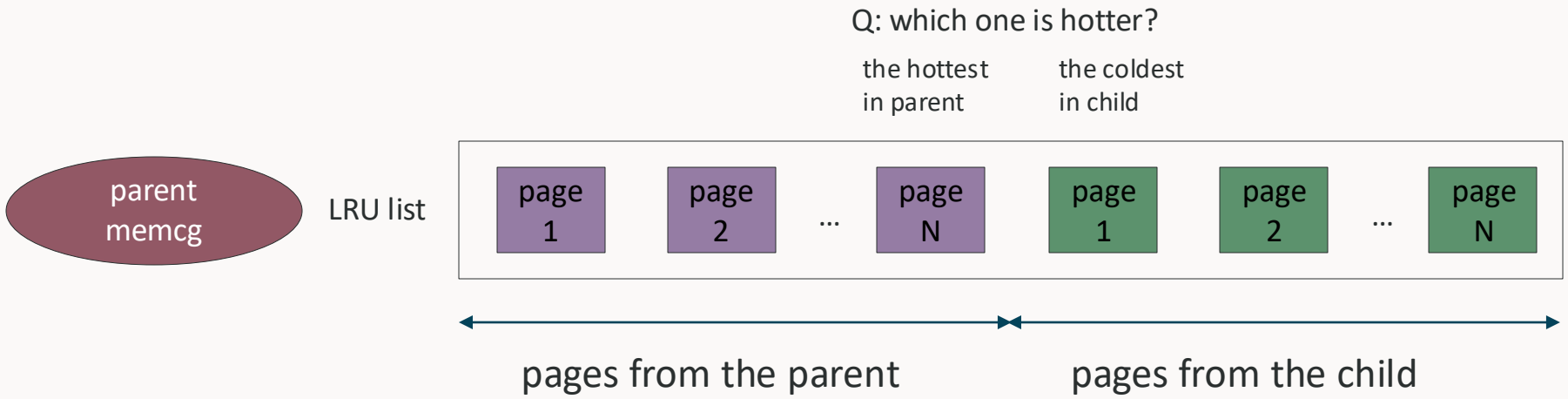
Merging two LRU lists can cause LRU disorder



- Pages from the parent are suddenly considered “colder” compared to the child
- Basically we’re adding noise to the LRU mechanism upon reparenting
- Reclaim can be inefficient when reclaiming wrong pages due to reparenting

Copyright © 2025, Oracle and/or its affiliates |

Merging two LRU lists can cause LRU disorder



- LRU only tracks relative hotness within one list
- We never know how hot pages are compared to pages in other LRU lists

Copyright © 2025, Oracle and/or its affiliates |

Maybe we can implement a “smarter” reparenting, but…

Smart reparenting inevitaly adds a lot of complexity, definitely not as first step

- A smarter repreanting method could be implemented
  - e.g.) At least give a second chance to pages that were considered active in the child
- But iterating over all pages can cause soft lockup if done under LRU lock
- Deferred reparenting comes with own challenges & concurrency issues with other MM features

Copyright © 2025, Oracle and/or its affiliates |

The central question: how severely can LRU be distrubed in practice?

A theoretical adversarial workload would be:

- Create a cgroup where its processes touch a huge amount of files in a short period
- Delete the cgroup after the processes exit
- Repeat this over and over

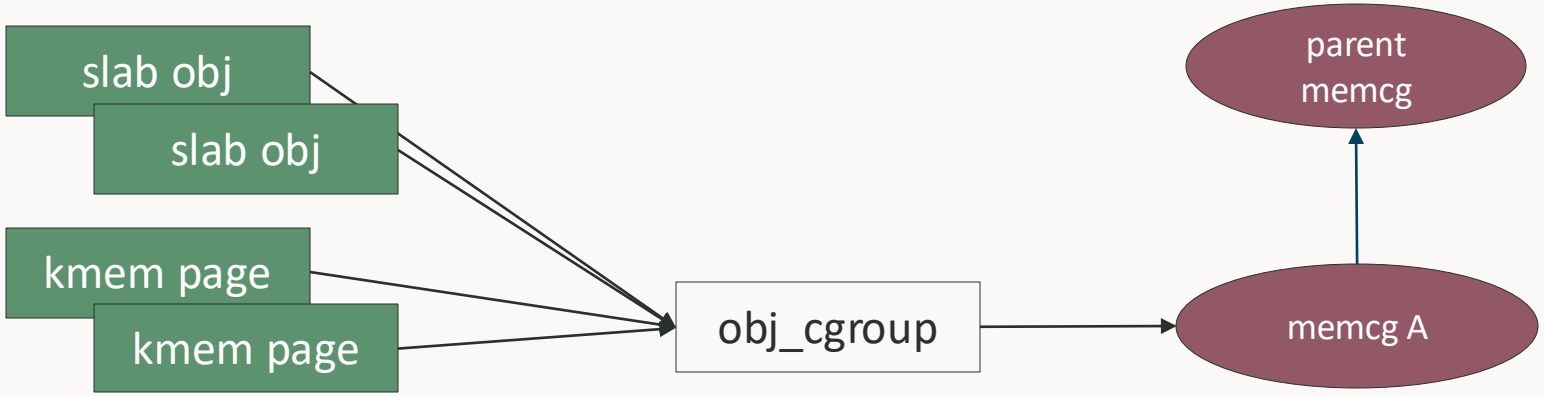
Does this sound realistic enough to consider more sophiscated method?

Copyright © 2025, Oracle and/or its affiliates |

Backup Slides

Quick Recap: obj_cgroup API

- Kernel pages are charged via obj_ cgroup API and they are reparented to the parent when a memcg is offlined (thanks to Roman Guschin)
- Let's eliminate the zombie memory cgroup problem: charge LRU folios with objcg, and reparent them on memcg offlining!
- The latest patch series posted on Oct 2025, by Qi Zheng & Muchun Song @ ByteDance



Copyright © 2025, Oracle and/or its affiliates |

Recharging vs. Reparenting

Recharging: charge the pages to other memory cgroups

- Tricky because we don’t know what other memcgs are accessing those pages
- Attempt by Yosry Ahmed @ Google in 2023 [1], but rejected by Johannes [2] because:
  - It is not consistent to apply reparenting for kernel memory and recharging for user memory
  - Unlike reparenting, recharging changes user-facing semantics: since resources belonging to a cgroup are charged to its all ancestors as well, it is changing the semantics to charge it to another cgroup other than its parent

[1] https://lore.kernel.org/lkml/20230720070825.992023-1-yosryahmed@google.com
[2] https://lore.kernel.org/lkml/20230720153515.GA1003248@cmpxchg.org

Copyright © 2025, Oracle and/or its affiliates |

Recharging vs. Reparenting

Reparenting: transfer pages from the child to the parent

- Not a perfect solution, but the most viable one so far
- Does not change user-facing semantics
- Reparenting fixes zombie memcg problem, but not the problem with incorrect accounting of shared resources
  - But the other problem is challenging to fix without introducing new problems

Copyright © 2025, Oracle and/or its affiliates |

Reparenting: LRU pages no longer pin its memory cgroup!

LRU pages are charged via objcg API and they pin objcg instead of memcg

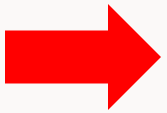
- folio_memcg() & folio_lruvec() users needs to if they want stable folio<->memcg binding
- folio_memcg() & folio_lruvec() users have to acquire RCU read lock
- folio<->memcg binding becomes stable only after acquiring LRU lock

```
struct lruvec *folio_lruvec_lock(struct folio *folio)
{
        struct lruvec *lruvec = folio_lruvec(folio);

        spin_lock(&lruvec->lru_lock);
        lruvec_memcg_debug(lruvec, folio);

        return lruvec;
}
```

w/o reparenting



```
struct lruvec *folio_lruvec_lock(struct folio *folio)
{
        struct lruvec *lruvec;

        rcu_read_lock();
retry:
        lruvec = folio_lruvec(folio);
        spin_lock(&lruvec->lru_lock);
        if (unlikely(lruvec_memcg(lruvec) != folio_memcg(folio))) {
                spin_unlock(&lruvec->lru_lock);
                goto retry;
        }

        return lruvec;
}
```

w/ reparenting

Copyright © 2025, Oracle and/or its affiliates |


ORACLE